

BRAINWARE UNIVERSITY

ML PROJECT-BASED LEARNING REPORT (WEEK 5)

1. Basic Information

Title of the Project:	Text Encryption & Decryption Tool and Word Frequency Counter (Program 1 & 2)
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS
Project Date	07/08/2026

2. Introduction and Background

Week 5 focuses on two console-based text-processing utilities: a **Text Encryption & Decryption Tool** using a character-shift cipher, and a **Word Frequency Counter** for analysing words and characters. These programs introduce text transformation, ASCII arithmetic, cleaning, and tokenisation.

3. Objectives of the Project (Week 5)

Program 1 - Text Encryption & Decryption Tool

- Create a boxed console-based encryption/decryption tool.
- Implement a reversible printable-ASCII character-shift cipher.
- Use reusable `shift()`, `encrypt()`, and `decrypt()` functions.
- Verify that decrypted text matches the original message.

Program 2 - Word Frequency Counter

- Create a boxed console-based word and character counter.
- Remove punctuation and symbols before word splitting.
- Implement independent word and character counting.
- Test the tool with irregular spacing and punctuation.

4. Problem Statement / Driving Question

Program 1: How can text be transformed using a simple reversible character-shift algorithm while allowing exact recovery of the original message?

Program 2: How can raw text be cleaned and divided into words to produce accurate counts despite punctuation and spacing variations?

5. Methodology (Week 5 Activities)

Program 1 - Text Encryption & Decryption Tool

- Implemented `shift()` using printable ASCII codes (32–126) and modulo arithmetic.
- Used positive shifts for encryption and negative shifts for decryption.
- Added a continuous menu with error handling.
- Tested encryption and decryption to confirm accurate recovery.

Program 2 - Word Frequency Counter

- Reused the boxed console interface.
- Created `clean_text()` to replace unwanted symbols with spaces and split text into words.
- Implemented separate functions for word and character counting.
- Added a continuous menu and tested punctuation-heavy and irregularly spaced text.

6. Expected Outcomes / Deliverables (Week 5)

- Functional encryption/decryption and word-counting utilities.
- Clear, consistent boxed terminal interfaces.
- Understanding of ASCII arithmetic and text tokenisation.

7. Core Logic

Program 1 - Character-Shift Cipher

The `shift()` function is the core of the encryption tool. It converts each character to its ASCII code, adds (or subtracts, for decryption) a fixed offset, and wraps the result using modulo 95 so it stays within the printable ASCII range. Because `encrypt()` and `decrypt()` both call the same `shift()` routine with opposite signs, the cipher is fully symmetric and reversible.

```
def shift(text, amount):
    out = []
    for ch in text:
        code = ord(ch)
        if 32 <= code <= 126:           # shift only printable ASCII
            code = ((code - 32 + amount) % 95) + 32  # wrap within range
        out.append(chr(code))
    return ''.join(out)

def encrypt(text):
    return shift(text, SHIFT)

def decrypt(encoded):
    return shift(encoded, -SHIFT)      # reverse shift to recover text
```

Note: the shift amount is currently a fixed constant (`SHIFT = 5`) rather than a user-supplied key; making the shift key configurable at runtime is planned as a future enhancement.

Program 2 - Text Cleaning and Word Counting

The `clean_text()` and `counter_words()` functions together turn raw input into an accurate word count. `clean_text()` replaces every punctuation or symbol character with a space, and `counter_words()` then splits the cleaned text and discards any empty tokens produced by consecutive separators, giving a reliable word count regardless of formatting irregularities in the input.

```
def clean_text(text):
    for char in chars_to_remove:
        text = text.replace(char, " ")    # strip punctuation/symbols
    return text.split(" ")

def counter_words(text):
    word_list = [word for word in clean_text(text) if word != '']
    return len(word_list), word_list      # ignore empty tokens
```

Note: `clean_text()` currently strips a fixed, hard-coded list of punctuation characters; extending this to a configurable or locale-aware character set is a possible future enhancement.

8. Output Screenshot

Program 1 - Text Encryption & Decryption Tool

```

  _____
 |           |
 |  ENCRYPTOR  |
 |           |
 |_____     |_____
 |           |
 | Terminal Encryption (Character Shift) |
 |           |
 | [1] Encrypt text |
 | [2] Decrypt text |
 | [3] Exit         |
 |_____           |_____
 | > Enter option: 1 |
 | > Enter text to encrypt: XORDIYANS |
 |           |
 | ENCRYPTED:        |
 | ]TWIN^FSX        |
 |_____           |_____
 |           |
 | [1] Encrypt text |
 | [2] Decrypt text |
 | [3] Exit         |
 |_____           |_____
 | > Enter option: 2 |
 | > Enter encrypted text: ]TWIN^FSX |
 |           |
 | DECRYPTED:        |
 | XORDIYANS        |
 |_____           |_____
 |           |
 | [1] Encrypt text |
 | [2] Decrypt text |
 | [3] Exit         |
 |_____           |_____
 | > Enter option:  |

```

Program 2 - Word Frequency Counter

```

  _____
 |           |
 |  WORD COUNTER  |
 |           |
 |_____     |_____
 |           |
 | Word Frequency Counter |
 |           |
 | [1] Count words |
 | [2] Count characters |
 | [3] Exit        |
 |_____           |_____
 | > Enter option: 1 |
 | > Enter text: Xordiyans on the way |
 |           |
 | Total words: 4    |
 | Words:           |
 | Xordiyans, on, the, way |
 |_____           |_____
 |           |
 | [1] Count words |
 | [2] Count characters |
 | [3] Exit        |
 |_____           |_____
 | > Enter option:  |

```

9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language used to implement both console utilities
ord() / chr()	Converting characters to and from their ASCII code points for the shift-cipher
Modulo Arithmetic	Wrapping shifted character codes within the printable ASCII range (32-126)
str.replace() / str.split()	Stripping punctuation and splitting text into individual words for counting
List Comprehensions	Filtering out empty tokens when building the word list
Functions & Control Flow	Modularising the encrypt/decrypt routines and the word/character counting routines

10. References

- Python Official Documentation: <https://docs.python.org/3/>
- Python Built-in Functions (ord, chr): <https://docs.python.org/3/library/functions.html>
- Python String Methods Reference: <https://docs.python.org/3/library/stdtypes.html#string-methods>
- Brainware University - Python Learning Course Guidelines

11. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
